Profiles, Environments & Enterprise Use

1. How would you migrate a legacy non-Maven Java project to Maven step by step?

A: I would migrate in these simple steps:

1. **Create a pom.xml file** in the project's root folder and define the project's basic details (group ID, artifact ID).
2. **Add dependencies** by manually listing all the current libraries the project uses (like Spring, Hibernate, etc.) inside the <dependencies> section.
3. **Configure the build** by adding necessary plugins, like the maven-compiler-plugin, to ensure the code compiles correctly.
4. **Test and fix** any class path issues until the project successfully builds using mvn clean install.

2. How do Spring Boot starters simplify Maven dependency configuration in a Spring Boot project?

A: Spring Boot starters simplify configuration because they are **pre-packaged bundles** of related libraries. Instead of listing ten individual libraries needed for web development, you just list one starter (e.g., spring-boot-starter-web). The starter automatically pulls in all the correct, compatible dependencies and manages their versions for you.

3. What is a SNAPSHOT version in Maven, and how is it different from a release version?

A: A **SNAPSHOT** version means the dependency is currently under active development and is considered **unstable** or changing. A **release version** (e.g., 1.0.0) is considered **stable** and finished. Maven will constantly try to download new versions of a SNAPSHOT dependency, but it only downloads a release version once.

4. Explain a situation where you had to debug a complex dependency conflict using Maven. What steps did you take?

A: I had a conflict where two libraries needed different, incompatible versions of a shared tool (like Guava). I debugged it by running the command **mvn dependency:tree** to see which

library was bringing in the older, unwanted version. I then fixed it by using the **<exclusion>** tag to block the bad version from the dependency that was pulling it in.

5. Have you ever needed a custom Maven plugin? What problem did it solve, and how did you implement or configure it?

A: *A common example:* Yes, I configured a custom plugin to automate **code license header insertion**. The problem it solved was ensuring every new source file had the mandatory corporate copyright notice before the code was compiled. I configured it by defining the plugin's details in the pom.xml and binding its execution goal to the **validate** or **compile** phase of the build.